Introduction and Tutorial on Couple of Deep Learning and Disease Dynamics

Pengfei Song, Xi'an Jiaotong University

2026.07.09

Joint work with Prof Jianhong Wu (York University), Yanni Xiao (XJTU), Shuangshuang Yin (XJTU)

Mathematical Biology Class in York University

Chanllenges in Modelling Disease Spread: Unknown Mechanisms

Epidemic models have proved to be a very powerful tool in guiding public health measures, learning from the past and preparing for the future.

Nonetheless, modeling and controlling the emerging infectious disease such as COVID-19 remains a challenge due to the unknown mechanisms in transmission dynamics, for example,

- nonstandard incidence rate
- changing human mobility pattern
- wastewater early warning
- impact of misinformation on disease spreading (NLP, UDE, Epi Model)

精准建模难→全面拥抱数据驱动方法?

Accurate mechanistic modeling is hard → should we fully embrace data-driven methods?

Data driven methods to learn unknown mechanism

- Mechanisms can be characterized by ‘functions‘, ‘operators‘, ‘Distributions‘, ‘Stochastic Processes‘, ‘Manifolds‘. For example, $f(x) = \exp(x)$ describes exponential growth.
- How to learn mechanisms? Find the function or surrogates of the function from data. Approximation! $\exp(x) \approx 1 + x + x^2/2 + ...$ Think about Taylor expansion, Fourier expansion.
  Functions: Neural Networks,Kolmogorov-Arnold Networks, Random feature model (Reservior Computing, ELM), GPs, Kernel Methods (e.g., SVM), Polynomials, Decision Trees
  Operators: DeepOnets, Neural Operator,
  Distributions: GAN, VAE, Auto-regressive models, Normalizing Flows, Diffusion Models, Energy Based Models, Consistency models
  Stochastic Processes: infinitely deep bayesian neural network as neural SDE
- Other Techniques: Transfer Learning; Active Learning

Chanllenges in Modelling Disease Spread: Data Heterogeneity

Pure Data Driven Methods are restricted by data heterogeneities in epidemiology.

- noisy: (Bayesian inverse problems)
- partially observed，sparsity
- small, no experiments
- delay: functional differential equations
- privacy
- ......

数据异质性→机理模型变得更加重要

Data heterogeneity → mechanistic models become even more important

机理与数据融合演变成了基本的科研范式

Fusing mechanisms and data has become a basic research paradigm

Data and Mechanism driven Model

AI can not directly be used in small, noisy, partially observed disease data

Question: Can we embed data driven model in mechanism driven model to represent, learn and finally discover the unknown mechanisms?

Unknown formula of force of infection due to virus evolution, change human behaviour, impact of misinformation, ...

$$\begin{cases} \dfrac{\mathrm{dS}}{\mathrm{dt}} = -S\beta I, \\ \dfrac{\mathrm{dI}}{\mathrm{dt}} = S\beta I - \gamma I, \end{cases} \qquad \begin{cases} \dfrac{\mathrm{dS}}{\mathrm{dt}} = -S * (\text{Neural network}), \\ \dfrac{\mathrm{dI}}{\mathrm{dt}} = S * (\text{Neural network}) - \gamma I, \end{cases}$$

Universal Differential Equations

"Universal" means "universal approximators" (neural networks, GPs, SVM, random feature models, ...)

UDEs (proposed by Prof. Christopher Rackauckas, MIT, 2020(Christopher Rackauckas et.al. 2020 arxiv) are initial value problems with the following forms:

$$\frac{du}{dt} = f_{\theta_2}(u, t, \mathrm{UniversalApproximator}_{\theta_1}(u, t)),$$

where f is a known mechanism and $\mathrm{UniversalApproximator}$ denotes the missing or unknown terms, θ_1 and θ_2 are parameters of known mechanisms and UAs, respectively, which can be estimated simultaneously.

[1]Rackauckas C, Ma Y, Martensen J, et al. Universal differential equations for scientific machine learning[J]. arXiv preprint arXiv:2001.04385, 2020.

[2]Yin S, Wu J, Song P*. Optimal control by deep learning techniques and its applications on epidemic models[J]. Journal of Mathematical Biology, 2023, 86(3): 36.

[3]Song P, Xiao Y. Estimating time-varying reproduction number by deep learning techniques[J]. J Appl Anal Comput, 2022, 12(3): 1077-1089. (Dedicated to Prof Jibin Li on his 80th birthday).

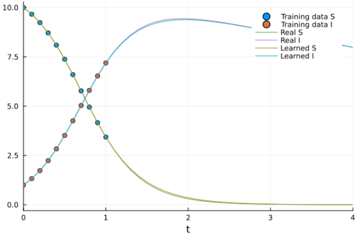
Universal ODE

Generate data from

$$\begin{cases} \frac{\mathrm{dS}}{\mathrm{dt}} = -S\beta\exp(-\alpha I)I^k, \\ \frac{\mathrm{dI}}{\mathrm{dt}} = S\beta\exp(-\alpha I)I^k - \gamma I, \end{cases} \tag{1}$$

Learn by

$$\begin{cases} \frac{\mathrm{dS}}{\mathrm{dt}} = -\mathrm{NeuralNetwork}(I)S, \\ \frac{\mathrm{dI}}{\mathrm{dt}} = \mathrm{NeuralNetwork}(I)S - \gamma I, \end{cases} \tag{2}$$



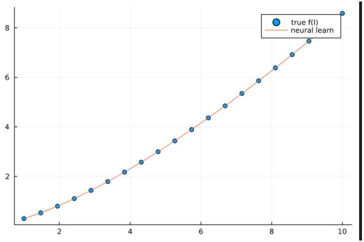
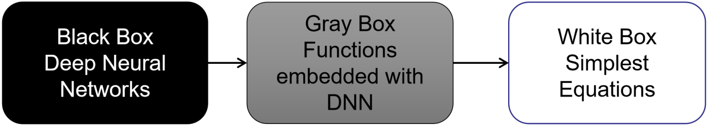


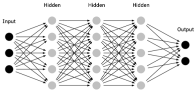
Figure: Using UDEs (2) to learn the nonstandard incidence rate in transmission model (1). Learn and generalize well

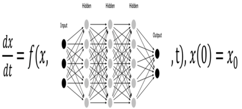
Black-box, Gray-box, White-box

How to recover the simplest function from deep neural networks?

Equation search methods.







$$\begin{cases} \dfrac{\mathrm{dS}}{\mathrm{dt}} = -S\beta\exp(-\alpha I)I^k, \\ \dfrac{\mathrm{dI}}{\mathrm{dt}} = S\beta\exp(-\alpha I)I^k - \gamma I, \end{cases}$$

[1] Song, Pengfei and Xiao, Yanni and Wu, Jianhong. (2023) Discovering first-principle behavior change transmission models by deep learning methods. One Chapter of Springer Book.

Equation Search Methods: Symbolic Regression

SR uses binary-tree to represent a function, and no particular formula is provided as a starting point. SR uses genetic programming, bayesian methods and deep learning methods to discover the simplest equations.

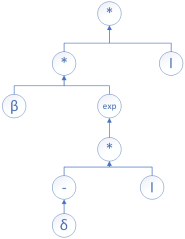


Figure: Expression tree of the function $f(I) = \beta \exp(-\delta I) I$

[1] Žegklitz J, Pošík P. Benchmarking state-of-the-art symbolic regression algorithms[J]. Genetic programming and evolvable machines, 2021, 22: 5-33.

Equation Search Methods: Sparse identification of nonlinear dynamic systems

SINDy applies a set of candidate functions $\mathbf{\Theta}(\mathbf{U})$ that would characterize the right-hand side of the governing equations, $u' = f(u) \approx \Theta(u)\Xi$, and estimate Ξ by sparse regression.

$$\begin{cases} \frac{\mathrm{dS}}{\mathrm{dt}} = -\beta SI, \\ \frac{\mathrm{dI}}{\mathrm{dt}} = \beta SI - \gamma I. \end{cases} \tag{3}$$

We choose the basic functions as

$$\mathbf{\Theta}([S, I]) = \left[S, I, SI, S^2 I, SI^2, S^2 I^2\right]$$

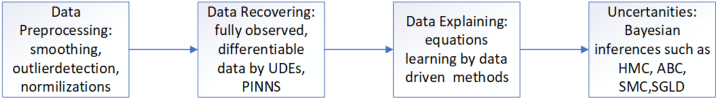
and now we use SINDy to discover the true equations.

[1] Brunton S L, Proctor J L, Kutz J N. Discovering governing equations from data by sparse identification of nonlinear dynamical systems[J]. Proceedings of the national academy of sciences, 2016, 113(15): 3932-3937.

Two-step Learning-Explaining Methods

UDE: representing and learning the unknown mechanisms by neural networks

Equation Search: recover the simplest function from neural networks



Remark: recover the simplest function from neural networks is favored by biologists and mathematician, but NOT in line with the philosophy of deep learning. Improve the generalization ability is.

[1] Song, Pengfei and Xiao, Yanni and Wu, Jianhong. (2023) Discovering first-principle behavior change transmission models by deep learning methods. One Chapter of Springer Book.

Outline

Example One: Estimating time vary reproduction number

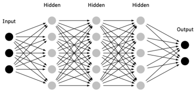
Represent effective reproduction number $\mathcal{R}_t$ as

$$\mathcal{R}_t = \mathrm{NeuralNetwork}_\theta(t, I), \tag{4}$$

and transmission model

$$\begin{cases} I' = \gamma \mathrm{NeuralNetwork}_\theta(t, I) I - \gamma I, \\ H' = \gamma \mathrm{NeuralNetwork}_\theta(t, I) I, \end{cases} \tag{5}$$

where $I(t)$ and $H(t)$ denote the number of infected individuals and accumulated confirmed cases at time t.

$\frac{dI}{dt} =$ 

$\times \gamma I - \gamma I$

[1] Song P, Xiao Y. Estimating time-varying reproduction number by deep learning techniques[J]. J Appl Anal Comput, 2022, 12(3): 1077-1089. (Dedicated to Prof Jibin Li on his 80th birthday).

Example One: Estimating time vary reproduction number

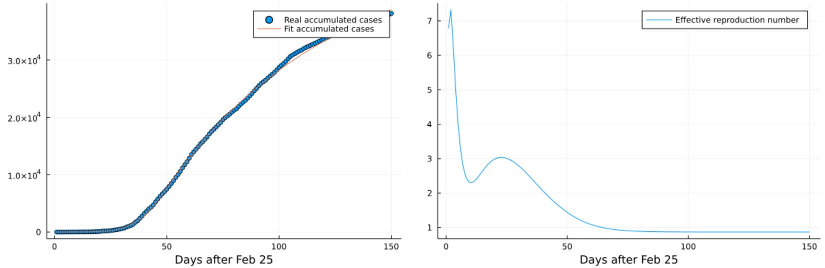
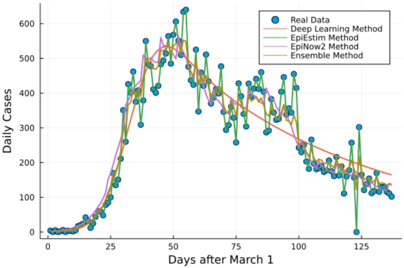


Figure: Left: Ontario's first wave COVID-19 case data. Right: effective reproduction number estimation by deep learning method.

Example One: Estimating time vary reproduction number



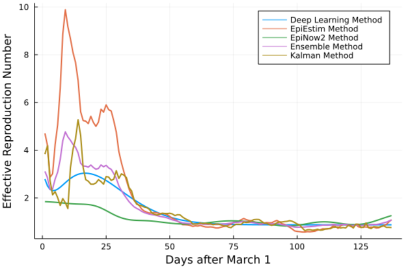


Figure: Left: Ontario's first wave COVID-19 case data fit by different methods. Right: effective reproduction number estimation by different methods.

Example One: Estimating time vary reproduction number

Methods	Data source	Smooth	Speed	Accuracy of $\mathcal{R}_t$
Deep Learning	Case data, infection period	strong	slow (3682s)	strong
State Space	Case data, infection period	weak	quick (<1s)	weak
EpiEstim	Case data, serial interval	weak	quick (<1s)	weak
EpiNow2	Case data, generation time, incubation period, delay distribution	normal	slow (2578s)	strong

Table: Comparison of different estimation methods: deep learning, state space, EpiEstim, EpiNow2 Methods. Smooth measures the data fitting abilities.

Example Two: Learning Unknown Human Behaviour Change Mechanisms

we will use the data of Ontario to fit the following neural differential equation model:

$$\begin{aligned}
\frac{\mathrm{dS}}{\mathrm{dt}} &= -\mathrm{abs}(NN(I,R))S/N, \\
\frac{\mathrm{dI}}{\mathrm{dt}} &= -\mathrm{abs}(NN(I,R))S/N - \gamma I, \\
\frac{\mathrm{dR}}{\mathrm{dt}} &= \gamma I, \\
\frac{\mathrm{dH}}{\mathrm{dt}} &= abs(NN(I,R))S/N,
\end{aligned}$$

where $NN(I, R)$ denotes neural network to learn the human behaviour change, and H denotes accumulated cases.

[1] Song, Pengfei and Xiao, Yanni and Wu, Jianhong. (2023) Discovering first-principle behavior change transmission models by deep learning methods. One Chapter of Springer Book.

Example Two: Learning Unknown Human Behaviour Change Mechanisms

To start with, learn the data by UDEs.

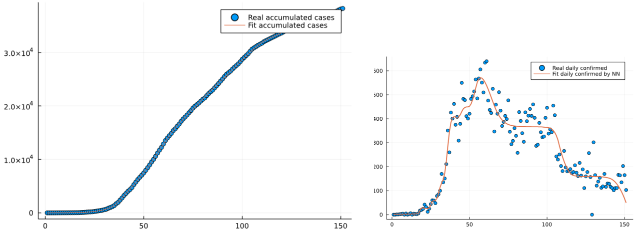


Figure: Learn Ontario first wave data by universal differential equations.

Example Two: Learning Unknown Human Behaviour Change Mechanisms

Use symbolic regression to find the simplest equation to fit $\mathrm{abs}(NN(I,R))$, and the equation found is kind of saturated function

$$\mathrm{abs}(NN(I,R)) \approx \frac{aI+b}{cR+d}.$$

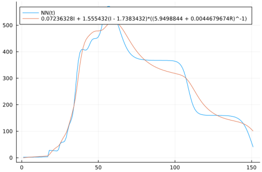


Figure: Symbolic regression to find the simplest equation to fit $\mathrm{abs}(NN(I,R))$.

[1]Song, Pengfei and Xiao, Yanni and Wu, Jianhong. (2023) Discovering first-principle behavior change transmission models by deep learning methods. One Chapter of Springer Book.

Example Three: Optimal Control

Consider the following optimal control problem in *Bolza* form:

$$\begin{cases} \max\limits_{u(t)\in\Omega(t)} J = \int_0^T g(x,u,t)dt + \phi(x(T),T) \\ s.t. \quad \frac{\mathrm{d}x}{\mathrm{dt}} = f(x,u,t), x(0) = x_0, \end{cases} \tag{6}$$

where the functions $f : \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R} \to \mathbb{R}^n$, $g : \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R} \to \mathbb{R}$ and $\phi : \mathbb{R}^n \times \mathbb{R} \to \mathbb{R}$ are assumed to be continuously differentiable. By representing the optimal control function $u(t)$ as a neural network

$$u(t) = \mathrm{NeuralNetwork}_\theta(t,x) \tag{7}$$

receiving t and x as inputs.

[1] Yin S, Wu J, Song P*. Optimal control by deep learning techniques and its applications on epidemic models[J]. Journal of Mathematical Biology, 2023, 86(3): 36.

Example Three: Optimal Control

$$\min_u \int_0^1 u^2 dt + I(1)^2, s.t. \quad I' = I - u, I(0) = 1.$$

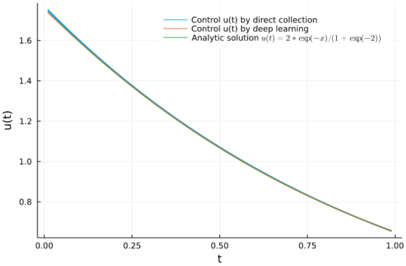
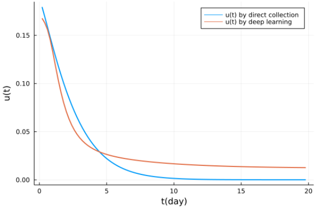


Figure: Left: control function. Right: state function.

Example Three: Optimal Control

$$
\begin{aligned}
\min \int_0^T & A * I(t) + u(t)^2 dt \\
s.t. \quad & S' = \Lambda - \beta S I - dS - u(t)S, \\
& E' = \beta S I - (d+\sigma)E, \\
& I' = \sigma E - (d+\gamma)I.
\end{aligned}
$$



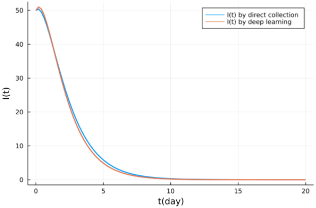


Figure: Left: control function. Right: state function. Comparison between direct allocation method and deep learning. Deep learning method find a good solution.

Example Three: Optimal Control

Methods	Direct method	Indirect method	HJB method	Deep learni
Transcriptions	Nonlinear programming problem (NLP)	Two point boundary value problem (TPBVP)	Dynamic programming	Parameter optimizatio
Trajectory or Parameter Optimization	Trajectory	Trajectory	Trajectory	Parameter
$u(x,t)$ or $x(u,t)$	-	$x(u,t)$	$u(x,t)$	$x(u,t)$
OtD or DtO	DtO	OtD	DtO	DtO and Ot
Using frequency	Most often	Often	Seldom	Seldom
Advantages	Mature Optimizers, easy to post, easy to solve	Accurate	Accurate	Flexible, extendable Bless of dimensional
Disadvantages	Less accurate	Hard to post, hard to solve, initial guess	Curse of dimensionality	Theoretical under exploring,

Outline

Books for Quickly Getting Started with Deep Learning

Deep Learning

- AI is enough
- Kevin Murphy, *Probabilistic Machine Learning*
  https://github.com/probml/pml2-book/releases (非常全，两册加起来三千页，可以当做字典用 / very comprehensive; the two volumes together are about 3000 pages and can be used as a dictionary)

Optimization

- 文再文等《最优化：建模、算法与理论》 / Zaiwen Wen et al., *Optimization: Modeling, Algorithms and Theory*
  http://faculty.bicmr.pku.edu.cn/~wenzw/optbook.html

Quickly Getting Started with Scientific Machine Learning

稀疏动力系统识别 / Sparse identification of dynamical systems

- Brunton S L, Proctor J L, Kutz J N. Discovering governing equations from data by sparse identification of nonlinear dynamical systems[J]. Proceedings of the national academy of sciences, 2016, 113(15): 3932-3937.

符号回归 / Symbolic regression: 会调用代码库就行，例如pysr / it is enough to know how to use existing packages, e.g., PySR
神经微分方程以及PINN / Neural differential equations and PINNs, physics-informed neural networks

- Neural DE: Kidger P. On neural differential equations[J]. arXiv preprint arXiv:2202.02435, 2022. (a PhD thesis)
- PINN: Lu L, Meng X, Mao Z, et al. DeepXDE: A deep learning library for solving differential equations[J]. SIAM review, 2021, 63(1): 208-228.

最好配合代码学!!!!!!
It is best to learn this together with code!!!!!!

Neural Networks

- input layer: $\mathcal{N}^0(\mathbf{U}) = \mathbf{U} \in \mathbb{R}^{d_{\mathrm{ta}}}$
- hidden layers: $\mathcal{N}^l(\mathbf{U}) = \sigma\left(W^l \mathcal{N}^{l-1}(\mathbf{U}) + b^l\right) \in \mathbb{R}^{N_l}$ for $1 \le l \le L-1$
- output layer: $\mathcal{N}^L(\mathbf{U}) = W^L \mathcal{N}^{L-1}(\mathbf{U}) + b^L \in \mathbb{R}^{d_{out}}$ where W^l is a Matrix or Tensor.

NN is composition of operators. Any abstract operator can be a layer, such as solution map of PDE, solution of implicit function. See more in NeurIPS 2020 tutorial: Deep Implicit Layers.

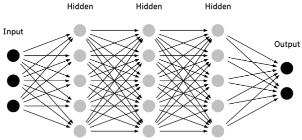


Figure: Scheme of deep neural network.

Neural Networks: Universal Apprimators of Functions

Neural Networks are universal apprimators. It can approximate unknown mappings and their derivatives. (Spectial "Taylor expansion", Difference: Projection VS Composition)

Theorem (Allan Pinkus 1999 Acta Numer)

Let $m_i \in \mathbb{Z}^{d+}, i = 1, \cdots, s$, *and set* $m = \max_{i=1,\cdots,s} |m_i|$. *Assume* $\sigma \in C(\mathbb{R}^d)$ *and that* σ *is not a polynomial. Then a single hidden layer neural network:*

$$\mathcal{M}(\sigma) := \textbf{span}\left\{\sigma(\mathbf{w} \cdot \mathbf{U} + b) : \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}\right\}$$

is dense in

$$C^{\mathbf{m}^1,\ldots,\mathbf{m}^s}\left(\mathbb{R}^d\right) := \cap_{i=1}^{s} C^{\mathbf{m}^t}\left(\mathbb{R}^d\right)$$

.

Neural Networks: Universal Apprimators of Operators

Neural Networks are universal apprimators. It can approximate unknown nonlinear operators in Banach space, such as ellptic, nonlocal diffussion.

Theorem (Lu lu et.al. 2021 Nat Mach Intell)

Suppose that X is a Banach space, $K_1 \subset X$, $K_2 \subset \mathbb{R}^d$ are two compact sets in X and $\mathbb{R}^d$, respectively, V is a compact set in $C(K_1)$. Assume that $G : V \to C(K_2)$ is a nonlinear continuous operator. Then, for any $\epsilon > 0$, there exist positive integers m, p, continuous vector functions $g : \mathbb{R}^m \to \mathbb{R}^p, f : \mathbb{R}^d \to \mathbb{R}^p$, and $x_1, x_2, \cdots, x_m \in K_1$, such that

$$|G(u)(y) - < g(u(x_1), u(x_2), \cdots, u(x_m)), f(y) >| < \epsilon$$

holds for all $u \in V$ and $y \in K_2$, where $< \cdot, \cdot >$ denotes the dot product in $\mathbb{R}^p$. Furthermore, the functions g and f can be chosen as diverse classes of neural networks, which satisfy the classical universal approximation theorem of functions, for example, (stacked/unstacked) fully connected neural networks, residual neural networks and convolutional neural networks.

Neural Networks: "人话"版总结 / Plain-Language Summary

- 神经网络就是一个强大的"多项式"。学习神经网络，就是学习原函数的"Taylor展开"；符号回归就是在找"Taylor展开"的原函数。
  A neural network is like a powerful "polynomial". Learning a neural network is like learning a "Taylor expansion" of the original function; symbolic regression tries to recover the original function behind that "Taylor expansion".

- $\mathrm{NeuralNetwork} = f(x, p)$, x是输入, p是参数(类比多项式的系数)
  $\mathrm{NeuralNetwork} = f(x, p)$, where x is the input and p denotes the parameters, analogous to polynomial coefficients.

Understanding Optimization from the Perspective of Dynamic Systems

- Optimization focuses on problems on the form

$$\min_{\theta} f(\theta)$$

- Optimization algorithm is actually find steady state of gradient flow

$$\begin{aligned}
\theta_{k+1} &= \theta_k - \eta_k \nabla f(\theta_k) && \text{(gradient descent)} \\
\frac{\mathrm{d}\theta}{\mathrm{dk}} &= -\nabla f(\theta). && \text{(gradient flow)} \\
\frac{\mathrm{d}^2\theta}{\mathrm{dk}^2} + \alpha \frac{\mathrm{d}\theta}{\mathrm{dk}} + \nabla f(\theta) &= 0 && \text{(Hamiltonian flow)}
\end{aligned}$$

How to Train Universal Differential Equations?

A simple example to understand how to estimate unknow parameter in ODE. Data: $\exp(1), \exp(2)$

$$\begin{cases} \min\limits_{\theta} J = (I(1) - \exp(1))^2 + (I(2) - \exp(2))^2 \\ s.t. \quad \dfrac{\mathrm{d}I}{\mathrm{dt}} = \theta I, t \in [0, 3]. I(0) = 1. \end{cases} \tag{8}$$

$\Phi_\theta(t) : \mathbb{R}^n \to \mathbb{R}^n$ be solution map. $I(t) = \Phi_\theta(t) I_0 = \exp(\theta t)$

$$\begin{aligned} &\min_{\theta} J = F(\theta) = (\Phi_\theta(1) I_0 - \exp(1))^2 + (\Phi_\theta(2) I_0 - \exp(2))^2 \\ &= (\exp(\theta) - \exp(1))^2 + (\exp(2\theta) - \exp(2))^2 \end{aligned} \tag{9}$$

Gradient flow: $\frac{\mathrm{d}\theta}{\mathrm{dk}} = -\nabla F(\theta)$. Steady state: $\theta^* = 1$.

How to Train Universal Differential Equations?

For ODE case, its essence is to solve the following optimal control problem:

$$\begin{cases} \min_{\theta} J = \int_0^T g(x, \mathrm{NeuralNetwork}_{\theta}(t, x), t) dt + \phi(x(T), T) \\ s.t. \quad \frac{\mathrm{d}x}{\mathrm{d}t} = f(x, \mathrm{NeuralNetwork}_{\theta}(t, x), t), x(0) = x_0, t \in [0, T]. \end{cases} \tag{10}$$

$\Phi_{\theta}(t) : \mathbb{R}^n \to \mathbb{R}^n$ be solution map. $x(t) = \Phi_{\theta}(t) x_0$

$$\min_{\theta} J = F(\theta) = \int_0^T g(\Phi_{\theta}(t)x_0, \mathrm{NeuralNetwork}_{\theta}(t, \Phi_{\theta}(t)x_0), t) dt + \phi(\Phi_{\theta}(T) \tag{11}$$

优化算法(例如Adam)求解过程，每一步迭代需要算一次$\frac{\mathrm{d}F}{\mathrm{d}\theta}$

During optimization algorithms such as Adam, each iteration requires computing $\frac{\mathrm{d}F}{\mathrm{d}\theta}$.

$$\begin{aligned}\frac{\mathrm{d}F(\theta)}{\mathrm{d}\theta} &= \int_0^T \left[g_x \frac{\mathrm{d}\Phi_\theta(t)x_0}{\mathrm{d}\theta} + g_u \left(\frac{\mathrm{dNeuralNetwork}_\theta}{\mathrm{d}\theta} \right.\right. \\ &\qquad \left.\left. + (\mathrm{NeuralNetwork}_\theta)_x \frac{\mathrm{d}\Phi_\theta(t)x_0}{\mathrm{d}\theta} \right) \right] dt. \end{aligned} \tag{12}$$

这涉及 $\frac{\mathrm{d}\Phi_\theta(t)x_0}{\mathrm{d}\theta}$的计算，是微分方程关于参数的求导的问题

This involves computing $\frac{\mathrm{d}\Phi_\theta(t)x_0}{\mathrm{d}\theta}$, namely differentiating the differential equation with respect to parameters.

How to Train Universal Differential Equations?

$\frac{\mathrm{d}\Phi_\theta(t)x_0}{\mathrm{d}\theta}$微分方程关于参数的求导. 把$\frac{\mathrm{d}\Phi_\theta(t)x_0}{\mathrm{d}\theta}$看成整体

This is the derivative of the differential equation with respect to parameters. Treat $\frac{\mathrm{d}\Phi_\theta(t)x_0}{\mathrm{d}\theta}$ as a whole.

$$\begin{cases} \frac{\mathrm{d}x}{\mathrm{dt}} = f(x, \mathrm{NeuralNetwork}_\theta(t, x), t), \\ \frac{\mathrm{d}\left(\frac{\mathrm{d}\Phi_\theta(t)x_0}{\mathrm{d}\theta}\right)}{\mathrm{dt}} = f_x \frac{\mathrm{d}\Phi_\theta(t)x_0}{\mathrm{d}\theta} \\ + f_u \left(\frac{\mathrm{dNeuralNetwork}_\theta}{\mathrm{d}\theta} + (\mathrm{NeuralNetwork}_\theta)_x \frac{\mathrm{d}\Phi_\theta(t)x_0}{\mathrm{d}\theta}\right), \end{cases} \tag{13}$$

这需要求解一个$n*(N+1)$维数的ODE, n是状态空间维数, N是神经网络维数

This requires solving an ODE of dimension $n*(N+1)$, where n is the state-space dimension and N is the neural-network dimension.

计算复杂度: $\mathcal{O}(N)$ / Computational complexity: $\mathcal{O}(N)$

Adjoint Methods in Abstract Space

$\min f(x) = f(x(\theta)) = F(\theta) \quad s.t. \quad c(x,\theta) = 0, c(\cdot,\cdot) : \mathbb{X} \times \mathbb{R}^N \to \mathbb{Y}$

若直接求导, 需要求一个维数为$N * \dim \mathbb{X}$的x_θ,一定得避开求x_θ

If we differentiate directly, we need to compute x_θ of dimension $N * \dim \mathbb{X}$; we must avoid computing x_θ.

Lagrange function: $L = f(x,\theta) + < c(x,\theta), \lambda >$

$$\begin{aligned}\frac{\mathrm{d}F}{d\theta} &=< \nabla_x c * x_\theta + c_\theta, \lambda > + < \nabla_x f, x_\theta > \\ &=< \nabla_x f + (\nabla_x c)^* \lambda, x_\theta > + < c_\theta, \lambda >\end{aligned}$$

$\nabla_x c, \nabla_x f$ are Frechet derivatives. $(\nabla_x c)^*$ is adjoint operator of $\nabla_x c$.

$$\frac{\mathrm{d}F}{d\theta} =< c_\theta, \lambda >$$

$$\nabla_x f + (\nabla_x c)^* \lambda = 0 \quad (\text{adjoint}, \dim \mathbb{Y}), \quad c(x,\theta) = 0 \quad (\text{forward}, \dim \mathbb{X})$$

共轭避开求x_θ之后，要求解$\dim \mathbb{X} + \dim \mathbb{Y}$维的未知量

After using the adjoint method to avoid x_θ, we only solve unknowns of dimension $\dim \mathbb{X} + \dim \mathbb{Y}$.

Adjoint Methods in Abstract Space

以ODE为例子(可以推广到SDE,PDE,DDE,就需要求解共轭算子$(\nabla_x c)^*$)

Take ODEs as an example; this can be extended to SDEs, PDEs, and DDEs by deriving the adjoint operator $(\nabla_x c)^*$.

$$J(\theta) = \int_0^T e(x,t)dt := f(x,\theta), c(x,t) := x' - m(x,\theta,t) = 0, x(0) = x_0,$$

$$\nabla_x f = -e_x, \nabla_x c = \frac{\mathrm{d}}{\mathrm{dt}} - m_x, (\nabla_x c)^* = -\frac{\mathrm{d}}{\mathrm{dt}} - m_x$$

最终避开了求x_θ

Finally, we avoid computing x_θ.

$$\begin{cases} \dfrac{\mathrm{d}J}{\mathrm{d}\theta} = \displaystyle\int_0^T \lambda(t) m_\theta dt, \\ \lambda'(t) = -e_x - m_x \lambda(t), \lambda(T) = 0, \\ x' = m(x,\theta,t), x(0) = x_0. \end{cases}$$

How to Train Universal Differential Equations?

"backpropagation" for differential equations: Adjoint sensitivity analysis in optimal control theory. Back to Pontryagin. 只需要求解一个$2n$维数的ODE, n是状态空间维数. 从$\mathcal{O}(N)$ 降到了$\mathcal{O}(1)$.

Only an ODE of dimension $2n$ needs to be solved, where n is the state-space dimension. The complexity is reduced from $\mathcal{O}(N)$ to $\mathcal{O}(1)$.

Theorem

Let

$$J(\theta) = \int_0^T e(x,t)dt, x' = m(x,\theta,t), x(0) = x_0,$$

where $\theta \in \mathbb{R}^k$ *and the functions* $m : \mathbb{R}^n \times \mathbb{R}^k \times \mathbb{R} \to \mathbb{R}^n$, $e : \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R} \to \mathbb{R}$ *are continuously differentiable. Then we have*

$$\begin{cases} \frac{\mathrm{d}J}{\mathrm{d}\theta} = \int_0^T \lambda(t) m_\theta dt, \\ \lambda'(t) = -e_x - m_x \lambda(t), \lambda(T) = 0, \\ x' = m(x,\theta,t), x(0) = x_0. \end{cases} \tag{14}$$

[1]Cao Y, Li S, Petzold L, et al. Adjoint sensitivity analysis for differential-algebraic equations: The adjoint DAE system and its numerical solution[J]. SIAM journal on scientific computing, 2003, 24(3): 1076-1089.

[2]Yin S, Wu J, Song P*. Optimal control by deep learning techniques and its applications on epidemic models[J]. Journal of Mathematical Biology, 2023, 86(3): 36.

Practical Engine: Scientific Machine Learning

The success behind machine learning is everything can and should be auto differentiable.

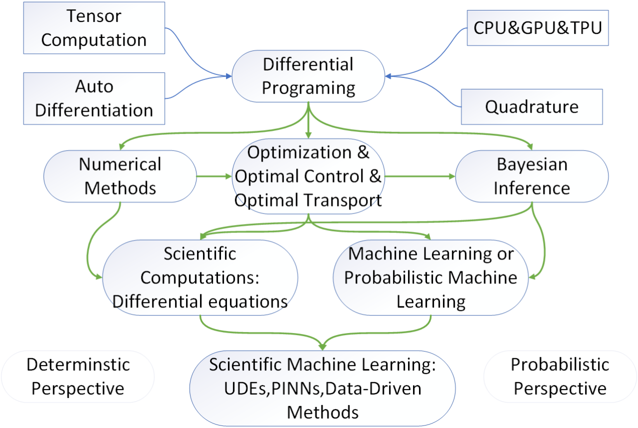


Figure:

Summary

- Couple of deep learning and disease dynamics: universal differential equations

$$\frac{du}{dt} = f_{\theta_2}(u, t, \mathrm{UniversalApproximator}_{\theta_1}(u, t)),$$

- Training Algorithms: optimal control, Adjoint sensitivity analysis
- Interpretability: equation search methods (Symbolic regression; Sparse identification of nonlinear dynamic systems), two-step learning explaining framework.

Thanks!